

UNIT 4

Tables: Rectangular tables - Jagged tables – Inverted tables - Symbol tables – Static tree tables - Dynamic tree tables - Hash tables.

Sorting techniques: Internal sorting - Insertion Sort, Selection Sort, Shell Sort, Bubble Sort, Quick Sort, Heap Sort, Merge Sort, Radix Sort.

External sorting: External storage devices – Sorting with tapes and disks.

THE SYMBOL TABLE:

An essential function of a compiler is to record the identifiers and the related information about its attributes types (numeric or character), its scope (where in the program it is valid) and in the case of procedure or the function names, such things as the number and types of its arguments, the mechanism of passing each argument and the type of result it returns.

A symbol table is a set of locations containing a record for each identifier with fields for the attribute of the identifier. A symbol table allows us to find the record for each identifier (variable) and to store or retrieve data from that record quickly.

For example, later an expression written in c such as `int x,y,z;` after compiling this statement, the symbol table entry will be

Memory location		
1	x	→ Location
2		→
3	y	→
	z	

Fig : symbol table

The first column of this table contains the entry of variables and the second contains the address of memory locations where values of these variables will be stored.

The way to implement symbol tables are :

1. Static tree table
2. Dynamic tree table.

On the first case, identifiers are known in advance and no deletions or insertions are allowed.

On the second case is the one in which identifiers are not known in advance.

STATIC TREE TABLE:

Symbol tables with property that, identifiers are known in advance and no additions or deletions are performed are called static.

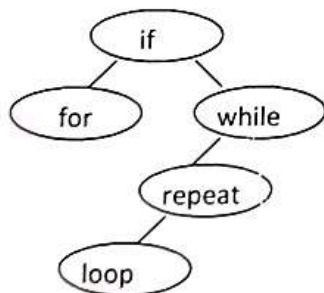
One solution is to sort the names and store them sequentially using binary search tree.

BINARY SEARCH TREE:

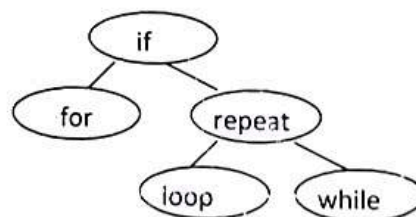
A binary search tree T is a binary tree, either it is empty or each node in the tree contains an identifier and

1. All identifier in the left sub-tree of T are less (numerically and alphabetically) than the identifier in the root node T
2. All identifier in the right sub-tree of T are greater (numerically and alphabetically) than the identifier in the root node T
3. The left and right sub-trees of T are also binary search trees.

For a given set of identifiers several binary tree are possible. The following fig. shows two possible binary search trees for a sub set of reserved words.



(a)



(b)

Fig 1: Two possible binary search trees

SEARCH FOR A KEY IN BST:

To determine whether an identifier X is present in a BST, X is compared with the root. If X is less than the identifier in the root, then the search continues in the left sub tree, the search terminates successfully, otherwise the search continues in the right sub tree.

ALGORITHM:

Procedure SEARCH (T,X,i)

// search the bst T for X. each node has fields LCHILD, IDENT, RCHILD. Return I = 0 if X is not in T. Otherwise, return I such that IDENT (i) = X. //

1. $i \leftarrow T$
2. while $i \neq 0$ do
3. case
4. : $X < \text{IDENT}(i)$: $I \leftarrow \text{LCHILD}(i)$ //search left sub-tree//
5. : $X = \text{IDENT}(i)$: return
6. : $X > \text{IDENT}(i)$: $I \leftarrow \text{RCHILD}(i)$ // search right child//
7. end
8. end
9. end SEARCH

On searching for an identifier at level k using algorithm SEARCH, k iterations of the which loop of lines 2-8 are made. Since this loop determines the cost of the search, it is reasonable to use the level no. of a node on its cost.

Consider the two search trees of the above figure. As names are encountered a match is search for in the tree. The second binary tree requires at most three comparisons to decide whether there is a match. The first binary tree may require four comparisons.

On evaluation binary search tree, it is useful to add a special 'square' node at every place there is a null link. Doing this to the tree of fig: 1 yields the tree of fig: 2.

Every binary tree with a node has $n+1$ null links and hence it will have $(n+1)$ square nodes. We shall call these nodes external nodes because they are not part of the original tree. The remaining nodes will be called internal nodes. Each time a binary search tree is examined for an identifier which is not in the tree, the search terminates at an external node. Since all searches represent unsuccessful search, external nodes will also be refused to as failure nodes.

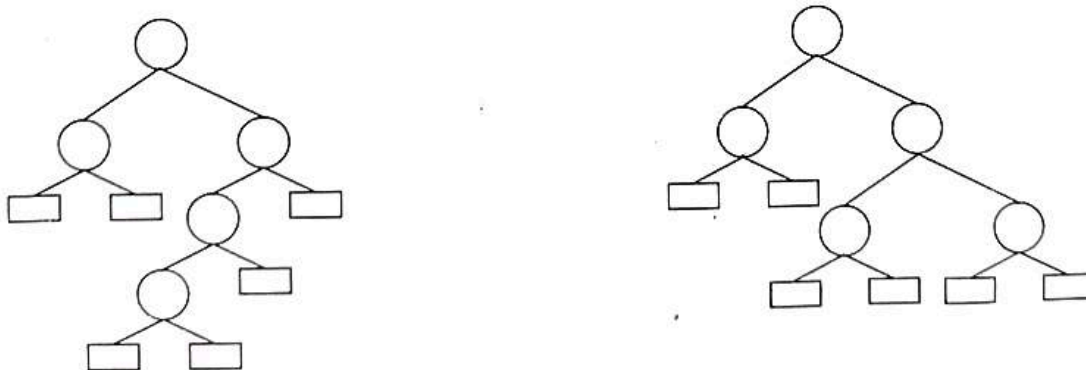


Fig: 2

A binary tree with external nodes added is an extended binary tree. Fig: 2 shows the extended binary trees corresponding to the search trees of fig: 1.

EXTERNAL PATH LENGTH:

Of a binary tree is the sum of the lengths of the path from the root to all external nodes.

INTERNAL PATH LENGTH:

Of a binary tree is the sum of the length of the path from the root node to all internal nodes.

For the tree of fig: 2(a), we obtain its internal path length I, to be

$$I = 0 + 1 + 1 + 2 + 3 = 7$$

Its external path length, E is

$$E = 2 + 2 + 4 + 4 + 3 + 2 = 17$$

This shows that the internal and external path lengths of tree with n internal nodes are related by the formula $E = I + 2n$.

Hence, binary trees with the max. E also have max. I. To obtain trees with minimal I, we must have as many internal nodes as close to the root as possible. We can have at most 2 nodes at distance 1, 4 at distance 2, and in general the smallest value for I is

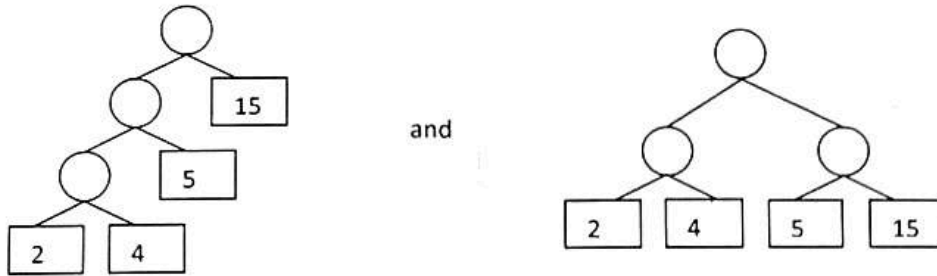
$$0 + 2 \cdot 1 + 4 \cdot 2 + 8 \cdot 3 + \dots = \sum_{(1 < k \leq n)} [\log_2 k] = O(n \log_2 n)$$

WEIGHTED EXTERNAL PETH LENGTH:

We are given a set of n+1 positive weight $q_1, \dots, q_{(n+1)}$. Exactly one of these weights is to be associated with each of the n+1 external nodes in a binary tree with n internal nodes.

The weighted external path length of such a binary tree is defined to be $\sum_{(1 \leq i \leq n+1)} [q_i k_i]$, where k_i is the distance from the root node to the external node with weight q_i . Now, problem is to determine a binary path length.

For example, suppose $n=3$ and we are given four weights: $q_1=15, q_2=2, q_3=4$ and $q_4=5$. Two possible trees would be



Their respective weighted external path lengths are :

$$2.3+4.3+5.2+15.1 = 43$$

and

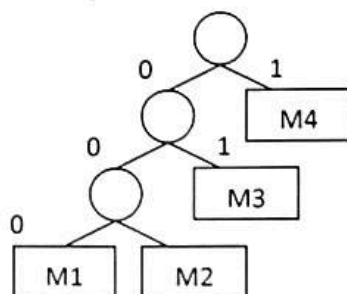
$$2.2+4.2+5.2+15.2 = 52$$

APPLICATION:(of binary trees with minimal weights external path length)

Binary trees with minimal weighted external path length find application in several access.

One application is to determine an optional merge pattern using 2 – way merge sort.

Another application of binary trees with minimal external path length is to obtain an optional set of codes for messages $M_1, \dots, M_{n+1}$. Each code is a binary string which will be used for transmission of the corresponding message. At the receiving end the code will be decoded using a decode tree. A decode tree is a binary tree in which external nodes represent messages. The binary list in the code word for a message determine the branching needed at each level of the decode tree to reach the correct external code. For example, if we interpret a zero as a left branch and a one as a right branch, then the decode tree



Corresponding to codes 000, 0001, 01 and 1 or messages M_1, M_2, M_3 and M_4 respectively. These codes are called Huffman codes. The cost of decoding a code word is proportional to the number of lists in the code. This number is equal to the distance of the corresponding external node from the root node.

If q_i is the relative frequency with which message M_i will be transmitted, then the expected decode time is $\sum(1 \leq i \leq n+1) [q_i d_i]$, where d_i is the distance of the external node for message M_i from the root node.

The expected decode is minimized by choosing code words resulting in a decode tree with minimal weighted external path length.

The solution to this is given by Huffman.

ALGORITHM HUFFMAN:

The algorithm HUFFMAN makes use of a list L of extended binary trees. Each node in a tree has three fields (WEIGHT, LCHILD and RCHILD). Initially, all trees in L have only one node. For each tree this node is an external node, and its weight is one of the provided q_i 's.

During the course of the algorithm, for any tree in L with root node T and depth greater than 1, $WEIGHT(T)$ is the sum of weights of all external nodes in T .

Algorithm HUFFMAN uses the sub-algorithm LEAST and INSERT. LEAST determines a tree in L with minimum WEIGHT and removes it from L . INSERT adds a new tree to the list L .

Procedure HUFFMAN(L, n)

// L is a list of n single node binary trees as described above //

for $I \leftarrow 1$ to $n-1$ do //loop $n-1$ times//

 call GETNODE (T) //create a new binary tree//

$LCHILD(T) \leftarrow LEAST(L)$ //by combining the trees//

$RCHILD(T) \leftarrow LEAST(L)$ //with two smallest weights//

$WEIGHT(T) \leftarrow WEIGHT(LCHILD(T)) + WEIGHT(RCHILD(T))$

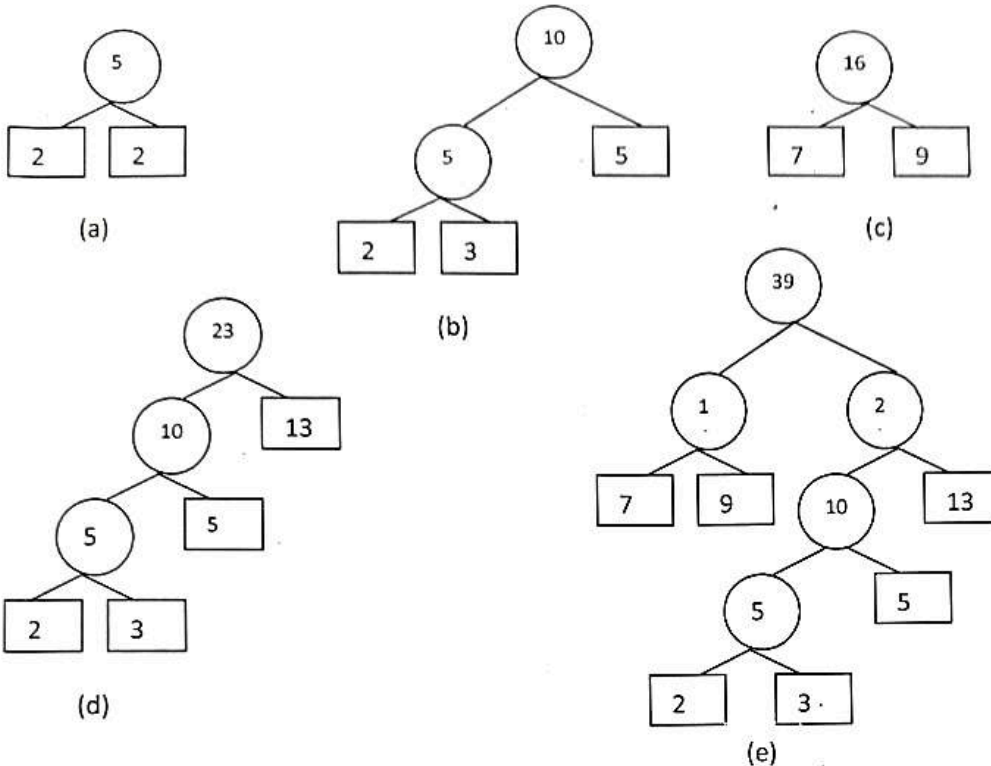
 Call INSERT (L, T)

end

end HUFFMAN

The way that this algorithm makes is given by an example shown below

Suppose we have given the weights $q_1=2, q_2=3, q_3=5, q_4=7, q_5=9$ and $q_6=13$. Then sequence of trees we would get is



The number in a circular node represents the sum of the weight of external nodes in that sub-tree.

The weighted external path length of the tree is

$$2.4 + 3.4 + 5.3 + 13.2 + 7.2 + 9.2 = 93$$

ANALYSIS:

The main loop is executed $n-1$ times. If L is maintained as a heap, then each call to LEAST and INSERT requires only $O(\log n)$ time.

[heap : is defined to be a complete binary tree with the property that the value of each node is at least as large as the value of its children nodes]

Hence, the computing time for the algorithm is $O(n \log n)$.

REPRESENTING SYMBOL TABLE AS A BINARY TREE:

If the binary search tree contains the identifiers $a_1, a_2, \dots, a_n$ with $a_1 < a_2 < \dots < a_n$ and the probability of searching for each a_i is p_i , then the total cost of any binary search tree is $\sum_{1 \leq i \leq n} (1 + \text{level } a_i) p_i$, when only successful searches are made.

Since unsuccessful searches, i.e. searches for identifiers set in the table, will also be made, we should include the cost of these searches in an cost measure.

Unsuccessful searches terminate with $i = 0$. Every node with a null sub-tree defines a point at which such a termination can take place.

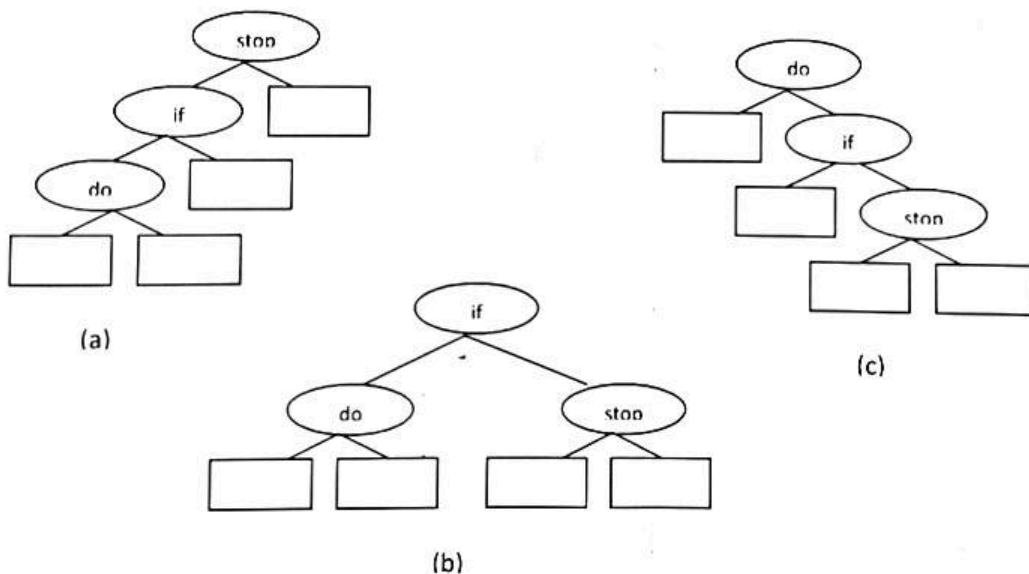
Let in place every null sub-tree by a failure node. If q_i the probability that the identifier being searched, then the cost of the failure node is $\sum_{(0 \leq i \leq n)} [q_i]$
 (level(failure node i) - 1)

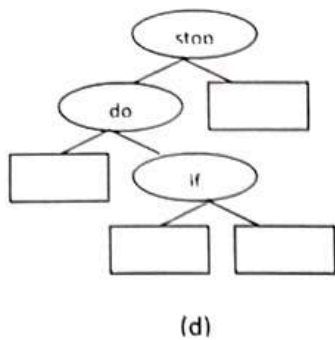
The total cost of a binary search tree is therefore

$$\sum_{(0 \leq i \leq n)} p_i (\text{level}(a_i)) + \sum_{(0 \leq i \leq n)} q_i (\text{level}(\text{failure node } i) - 1) \rightarrow \text{eq.1}$$

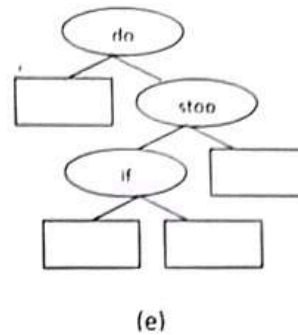
An optimal binary search tree for the identifier set $a_1, a_2, \dots, a_n$ is one which minimizes eq.1 over all possible set. Since all searches must terminate either successfully or unsuccessfully we have $\sum_{(1 \leq i \leq n)} p_i + \sum_{(1 \leq i \leq n)} q_i = 1$.

EXAMPLE : The possible binary search trees for the identifier set $(a_1, a_2, a_3) = (\text{do}, \text{if}, \text{stop})$ are :





and



With equal probabilities $p_i = q_i = 1/7$ for all i and j we have

$\text{cost}(\text{tree a}) = 15/7$; $\text{cost}(\text{tree b}) = 13/7$; $\text{cost}(\text{tree c}) = 15/7$; $\text{cost}(\text{tree d}) = 15/7$; $\text{cost}(\text{tree e}) = 15/7$. Tree b is optional.

DYNAMIC TREE TABLE:

Dynamic table may also be maintained as binary search trees. An identifier X may be inserted into a binary search tree T by using the search algorithm to determine the failure node corresponding to X . This gives the position in T where the insertion is to be made.

The following figure shows the binary search tree obtained by entering the months JANUARY to DECEMBER in that order into an initially empty binary search tree.

ALGORITHM:

Procedure BST (X, T, j)

//search the binary search tree T for the node j such that $\text{IDENT}(j) = X$. If X is not already in the table then it is entered at the appropriate point. Each node has LCHILD, IDENT and RCHILD fields//

$p \leftarrow 0; j \leftarrow T$ //p will trail j through the tree//

while $j \neq 0$ do

case

: $X < \text{IDENT}(j)$: $p \leftarrow j; j \leftarrow \text{LCHILD}(j)$ //search left sub-tree//

: $X = \text{IDENT}(j)$: return

: $X > \text{IDENT}(j)$: $p \leftarrow j; j \leftarrow \text{RCHILD}(j)$ //search right sub-tree//

end

```

end

//X is not in the tree and can be entered as a child of p//

call GETNODE (j) ; IDENT (j)  $\leftarrow$  X; LHCILD(j)  $\leftarrow$  0; RCHILD(j)  $\leftarrow$  0

case
    : T = 0 ; T  $\leftarrow$  j //insert into binary tree//
    : X < IDENT (p) : LCHILD (p)  $\leftarrow$  j
    : else : RCHILD (p)  $\leftarrow$  j
end
end
end BST
    
```

The maximum number of comparisons needed to search for any identifier of fig:3 is six for NOVEMBER. The average number of comparisons is $(1 \text{ for January} + 2 \text{ each for FEB \& MAR} + 3 \text{ each for APR, JUNE \& MAY} + \dots + 6 \text{ for NOV})/12 = 42/12 = 3.5$.

If the months are entered in the order JULY, FEB, AUG, DEC, MAR, OCT, APR, JAN, JUNE, SEP and NOV then the following tree is obtained.

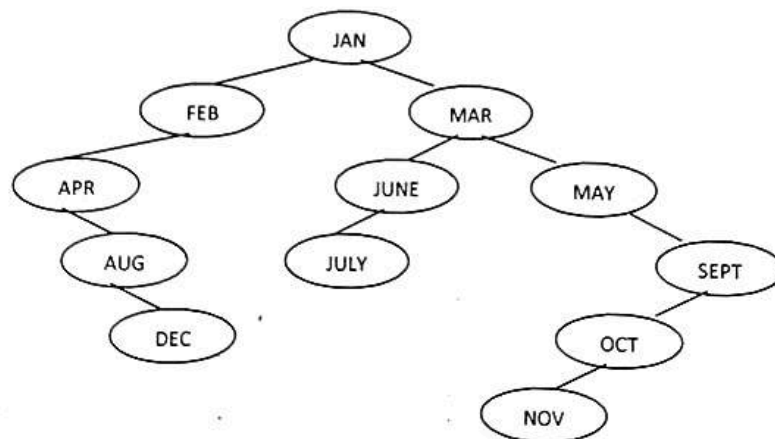


Fig : 3 Binary search obtained for the months of the year.

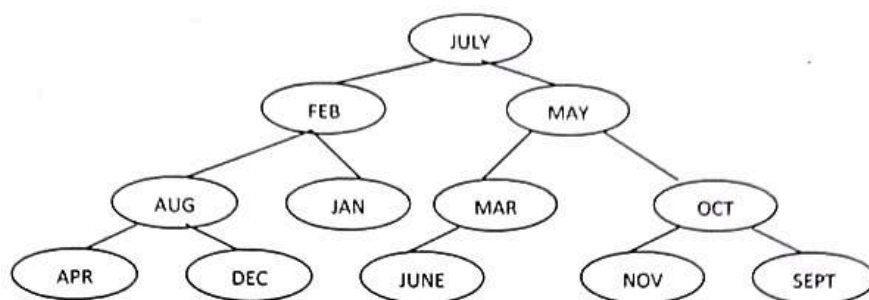


Fig : 4 Balanced tree for the months of the year.

On this fig :3 null links has six nodes on the path from root to NOV and only two nodes JAN & FEB an another path to a null link. But fig 4 is well balanced and the maximum number of identifier comparisons needed to find any identifier 4 and the average is $37/12 = 3.1$.

If identifier are entered in lexicographic order, the tree degenerates to a chain as in fig 5.

The maximum search time is now 12 identifier comparisons and the average is 6.5 and so sequential searching in an ordered file is the worst case.

Both the average and maximum search time will be minimized if the binary search tree is maintained as a complete binary tree at all times. However, since we are dealing with a dynamic situation, identifiers are being built and so it is difficult to achieve this ideal without making the time required to add new entries very high. Because, in some cases it is necessary to restructure the whole tree to accumulate the new entry and at the same time have a complete binary tree.

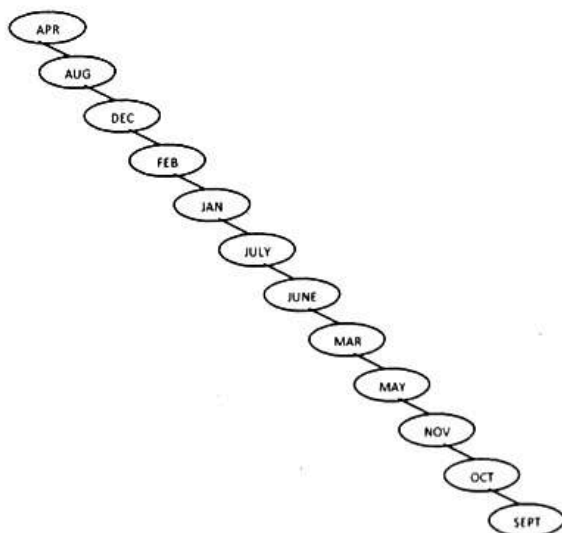


Fig : 5 Degenerate binary search tree

However, dynamic retrievals can be achieved using height balanced binary trees.

HEIGHT BALANCED TREE:

If T is a nonempty binary tree with T_L and T_R as its left and right sub trees, then T is height balanced if

- (i) T_L and T_R are height balanced and
- (ii) $|h_L - h_R| \leq 1$ where h_L and h_R are the height of T_L and T_R respectively.

i.e. The height of the left sub tree differs from the height of the right sub tree by no more than 1. An almost height balanced tree is called an AVL tree.

BUILDING HEIGHT BALANCED:

Each node on an AVL tree has the property that the height of the left sub tree is either one more, equal or one less than the height of the right sub tree. We may define a balance factor (BF) as

$$BF = (\text{Height of left sub tree} - \text{Height of right sub tree})$$

Further

If two sub-tree of same height, $BH = 0$

If right sub-tree is higher, $BF = -1$

If left sub-tree is higher, $BF = 1$

For example balance factor are stated near the nodes in fig 3. BF of the node is 0 because height of the right sub-tree and the height of the left sub-tree is three. The BF at the node DIN is 1 because the height of its left sub-tree is 2 and of right sub-tree is 1 etc.

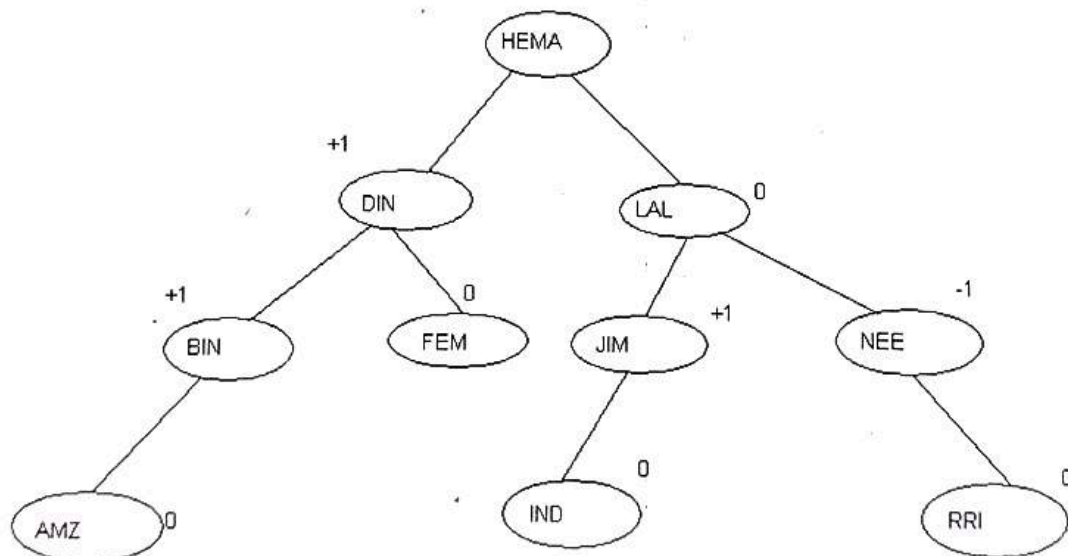


Fig 3 : Tree having a balance factor at each node

Let the values given were in the order BIN, FEM, IND, NEE, LAL, PRI, JIM, AMI, HEM, DIN and we needed to make the height balanced tree. It would work out as follows:

We begin at the root of the tree since the tree is initially empty we have



Fig 4(a)

We have FEM to be added. It would be on the already existing tree. Therefore we have

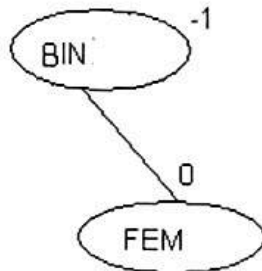


Fig 4(b)

The resulting tree is still height balanced. Now we need to add IND i.e. on the further right of FEM.

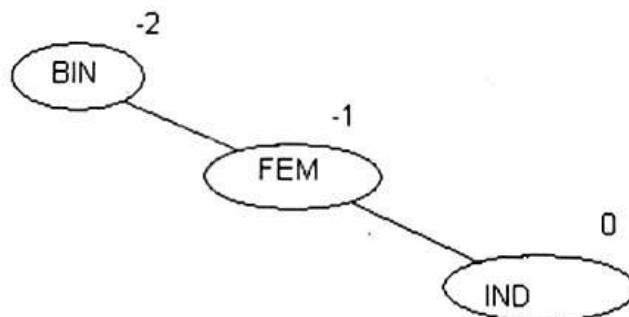


Fig 4 (c)

Since BF of one of the nodes is other than 0, +1, -1, we need to rebalance the tree. In such a case when the new node goes to the longer side, we need to rotate the structure counter clockwise i.e. a rotation is carried out in counter clockwise direction around the closest parent of the inserted node with BF = +2. In this case we get

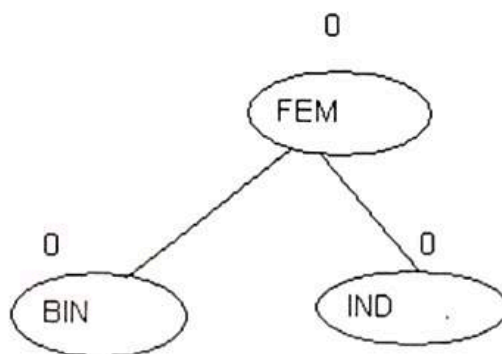


Fig 4(d)

We now have a balanced tree. On adding NEE, we get

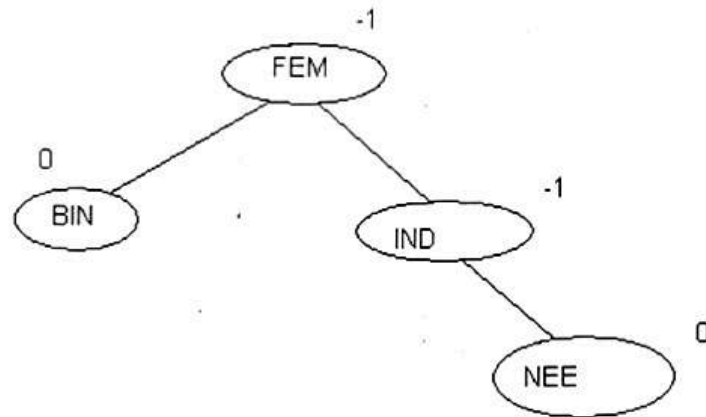


Fig 4(e)

Since all the nodes have $BF < \pm 2$ we continue with the next node. Now we need to add LAL

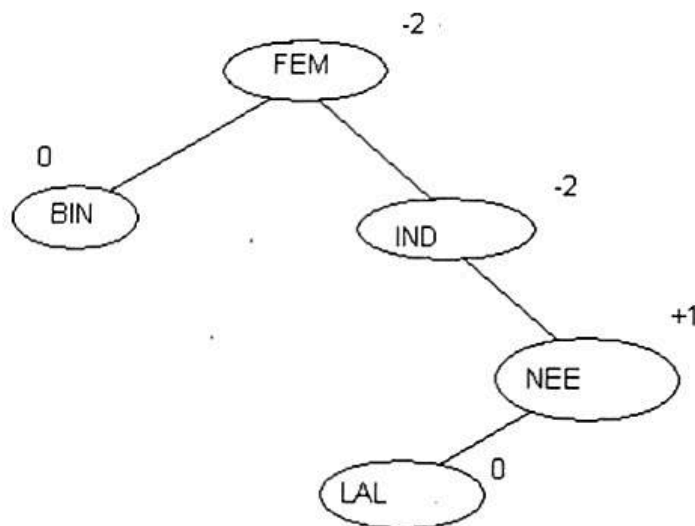


Fig 4(f)

To regain we need to rotate the tree counter clockwise at IND and we get

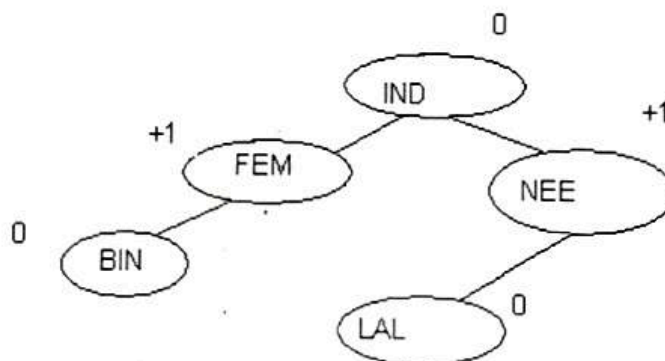


Fig 4(g)

On adding PRI we get

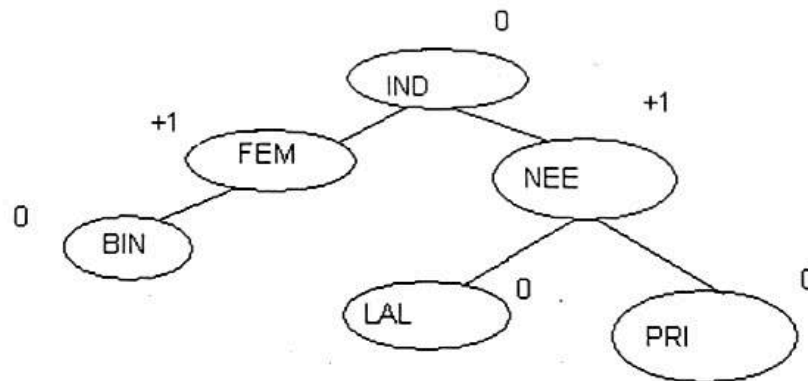


Fig 4(h)

On adding JIM, we get

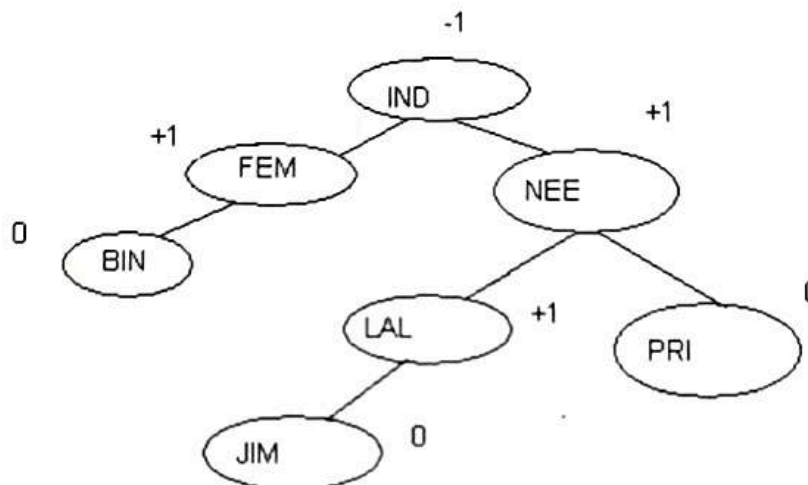


Fig 4(I)

The tree is still balanced. Now we add AMI

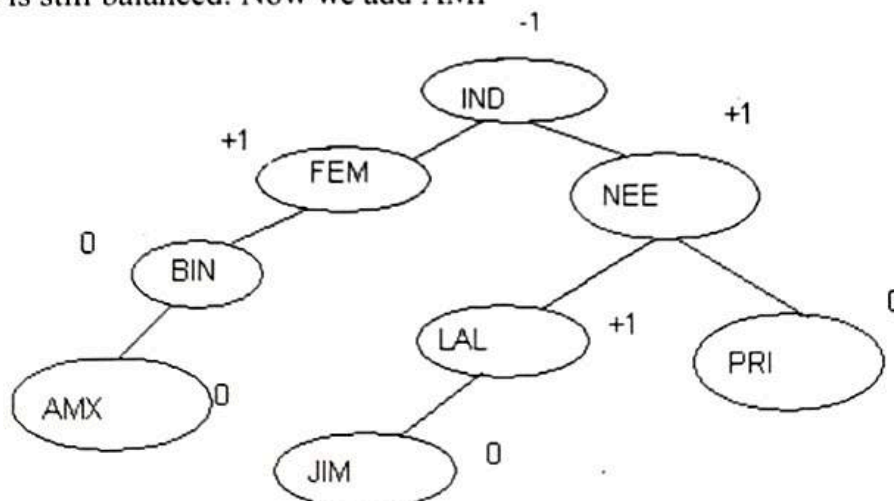


Fig 4(j)

Now we need to rotate the tree at FEN (i.e. the closest parent to AMI with BF = +2) in clockwise direction. On rotating it once we get

Now HEM is to be added. On doing so the structure we will get

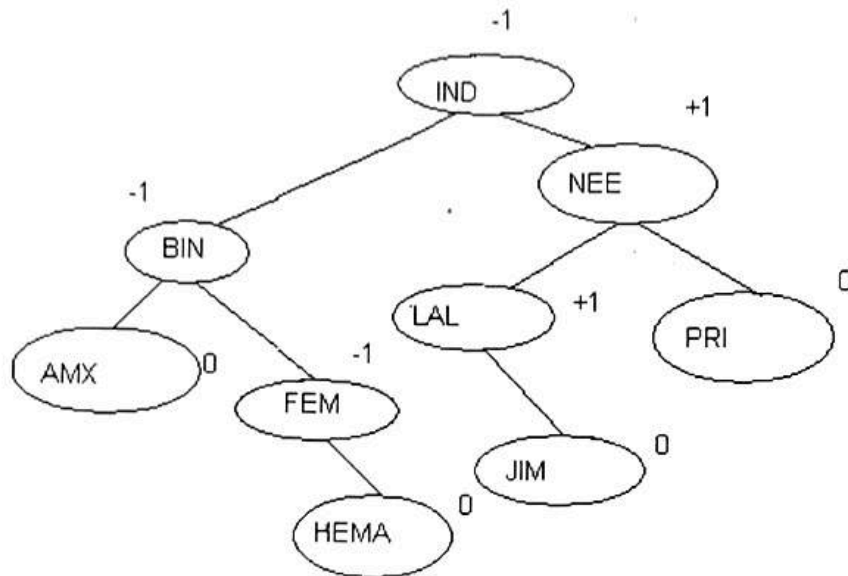


Fig 4(k)

Tree is still balance. Now we need to add DIN, we get

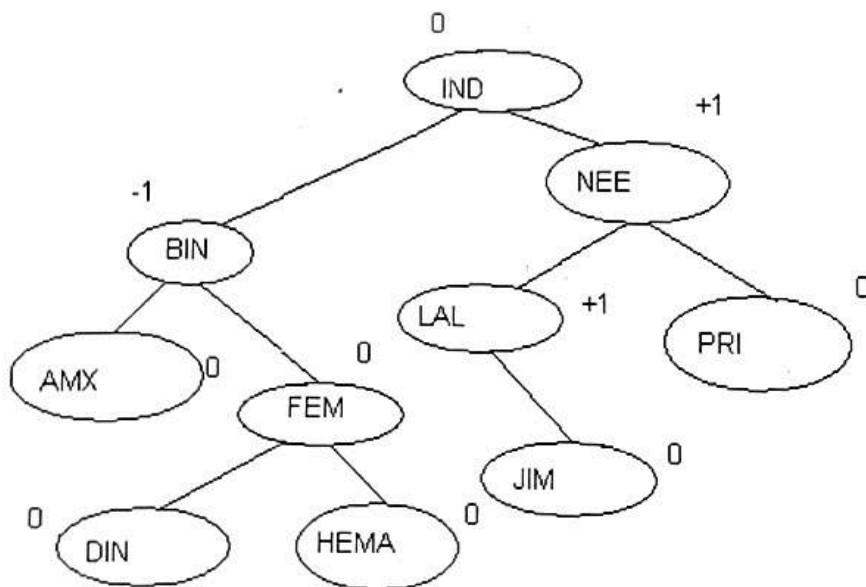


Fig 4(l)

Fig 4 : Construction of AVL tree

Let us take another example. Consider the following list of elements

3,5,11,8,4,1,12,7,2,6,10

The tree structure generated till we add 11 are all balanced structures as given below



Fig 5(a)

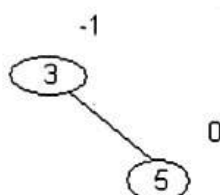


Fig 5(b)

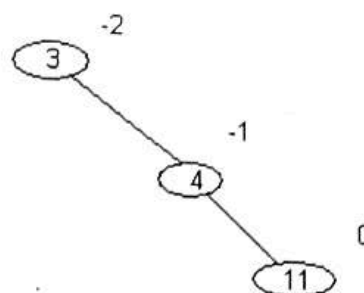


Fig 5(c)

Here we need re-balancing. Rotation around 3 would give

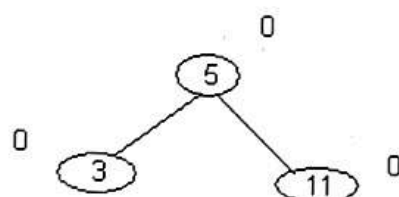


Fig 5(d)

Further we add 8, 4, 1, 12, 7, 2 as depicted in fig 5(a) and (k).

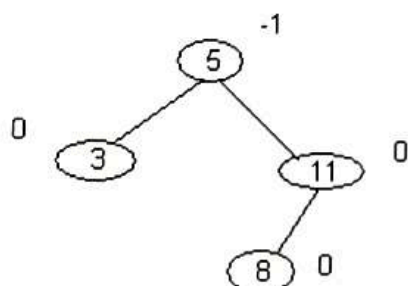


Fig 5(e)

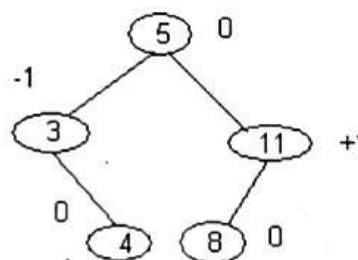


Fig 5(f)

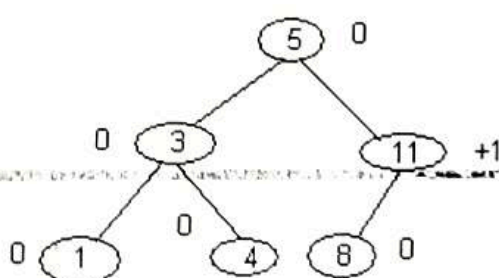


Fig 5(g)

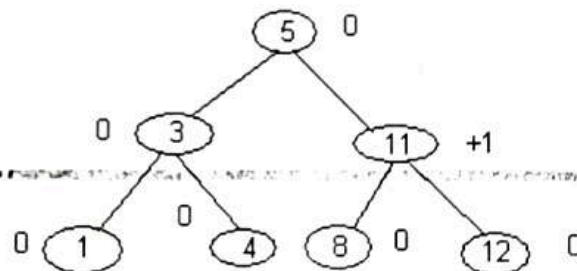


Fig 5(h)

After adding 6, the tree becomes unbalanced. We need to rebalance by rotating the structure at the node 8. It is shown in fig 5(i)

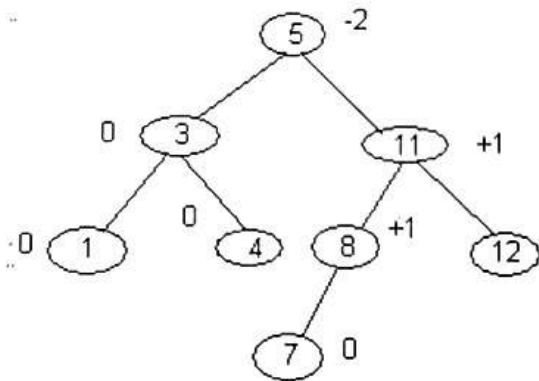


Fig 5(i)

Again on adding 10 we need to rebalance

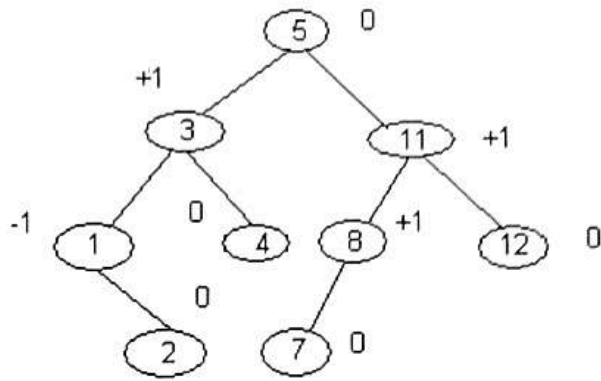


Fig 5(j)

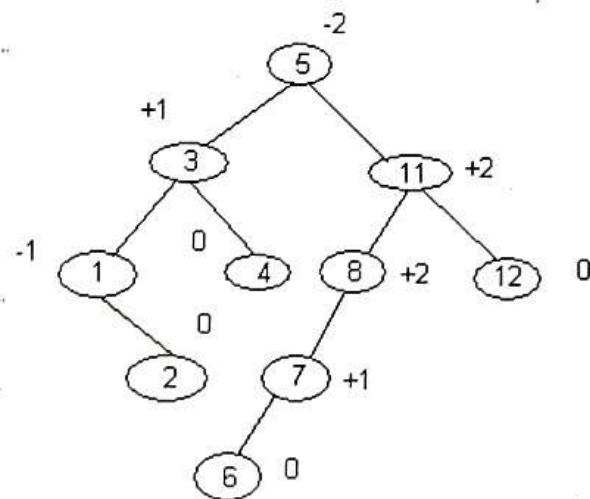


Fig 5(k)

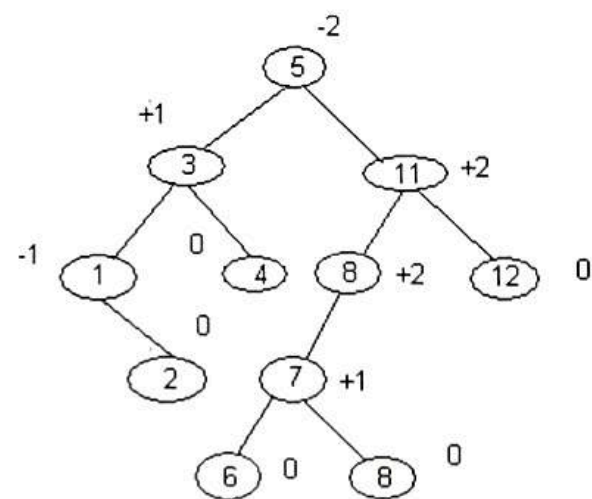


Fig 5(l)

Since the closest parent with BF = 2 is at node 11 we first rotate at it in clockwise direction. However, we would still not get a balanced structure as the right of node 7 shall have more nodes than the left of it. Therefore, we shall need another rotation, in anti-clockwise direction at node 7, and finally we get a structure as shown in fig 5(n).

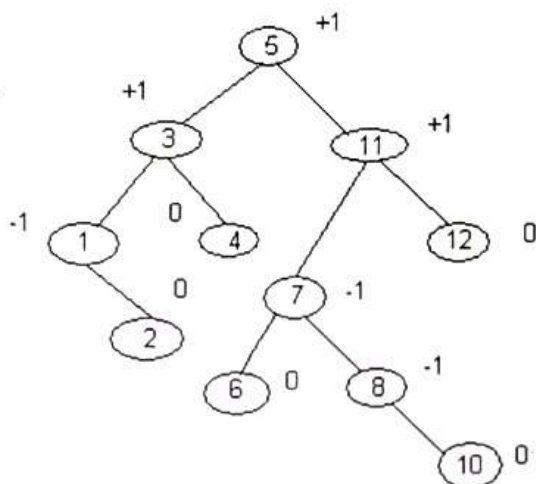


Fig 5(m)

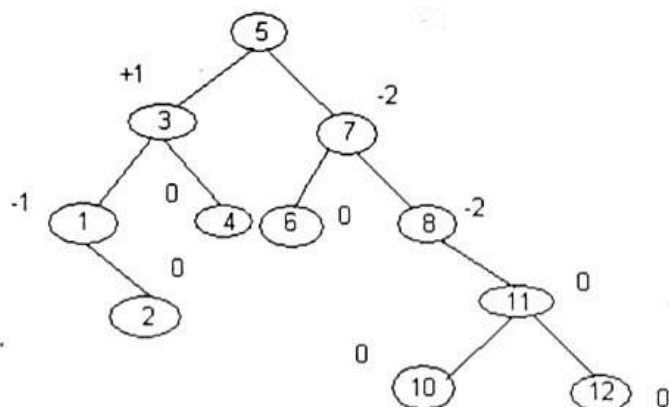


Fig 5(n)

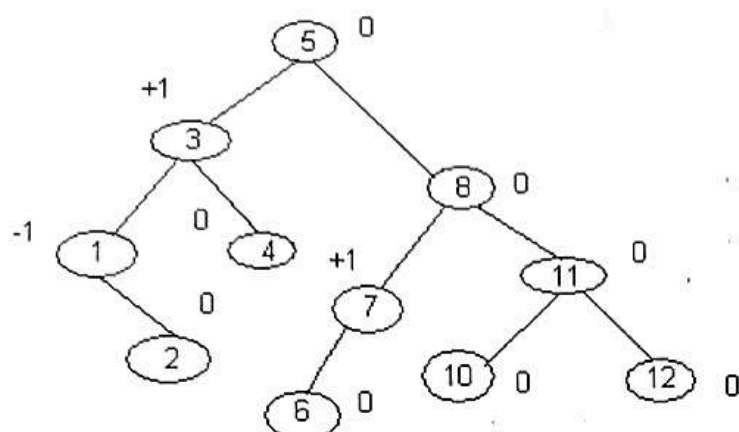


Fig 5(0)

Which is the height balance tree.

MULTIWAY TREE(M-TREE):

In a binary search tree, each node holds a single value and has at most two branches. Those in the left branch have less than the node value, while those in the right branch have values greater than the node value. This can be generalized by allowing more values at each node. For example, if we keep two values in each node, that means at most three branches, the descendants are split into three groups (maximum). The leftmost descendants will have values less than the first value in the root, the middle descendant will have values between the two values in the root node and the rightmost descendant will have values greater than the second value in the root node. Let us understand it more clearly by the following figure.

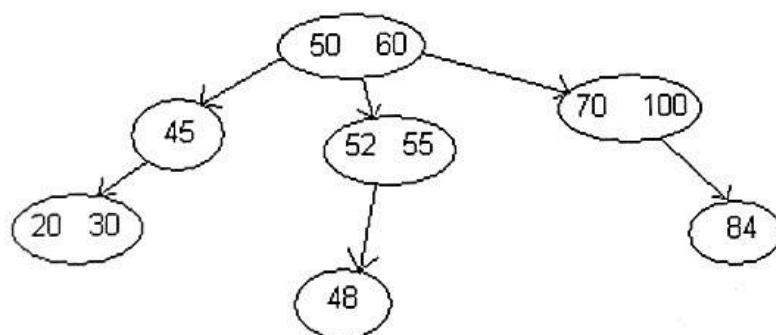


Fig : A 3-way tree

Note that all the nodes need not be full. Such a structure is called a multi way tree. A binary search tree is a two-way tree. The tree shown in fig above is a 3-way tree.

HASH TABLES:

The best search method, binary search technique involves number of comparisons, which has a search time of $O(\log_2 n)$. Another approach, is to compute the location of the desired record. The nature of this computation depends on the key set and the memory-space requirements of the desired record.

This key-to-address transformation problem is defined as a mapping or hashing function H , which maps the key space (K) into an address space (A).

The hash table is partitioned into b buckets. Each bucket is capable of holding s records. Thus, a bucket is said to consist of s slots, each slot being large enough to hold 1 record.

An overflow is said to occur when a new identifier I is mapped or hashed by function f into a full bucket.

Since, the key space is usually much larger than the address space, many keys will be matched to the same address. Such a many to one mapping results in collisions between records.

As an example, consider the hash table HT with $b = 26$ buckets, each bucket having exactly two slots, i.e. $s = 2$. The hash function f must map each of the possible identifiers into one of the numbers 1 – 26. Here, A-Z corresponds to the numbers 1-26 respectively, then the function f is defined by: $f(X)$ = the first character of X . The identifiers A,B,C... will be hashed into buckets 1,2,3...

The identifiers A, A1,A2 are synonyms. Take for eg., if A and A1 are already stored in the bucket. If A2 is to be stored then overflow occurs, since $s = 2$.

HASHING FUNCTIONS:

If X is an identifier chosen at random from the identifier space, then we want the probability that $f(X) = i$ to be $1/b$ for all buckets i . Then a random X has an equal chance of hashing into any of the b buckets. A hash function satisfying this property will be termed a uniform hash function.

Several kinds of uniform hash functions are in use.

(i) MID-SQUARE METHOD:

A key is multiplied by itself and the address is obtained by choosing an appropriate number of bits or digits from the middle of the square. The selection of bits or digits based on the table size and also they should fit into one computer word of memory.

Eg. Consider a key, 56789 and when it is squared we get 3224990521. If the three digit address is needed, then positions 5 to 7 may chosen, giving address 990.

(ii) DIVISION METHOD:

In this method, integer x is to divide by M and d then to use the remainder modulo M. The hash function is

$$H(x) = x \bmod M$$

Great care should be taken while choosing value for M and preferably it should be an even number. By making M a large prime number the keys are spread out evenly.

(iii) FOLDING METHOD:

A key is partitioned into a number of parts, each of which has the same length as the required address. The parts are then added together, ignoring the final carry, to form an address. For eg., if the key 356942781 is to be transformed into a three-digit address.

Two types:

1. Fold-shifting: 356, 942 and 781 are added to yield 079.
2. Fold-boundary method: 653, 942 and 187 are added together, yielding 782.

(iv) DIGIT ANALYSIS METHOD:

A hashing function referred to as digit analysis forms addresses by selecting and shifting digits or bits of the original key. For eg., a key 7546123 is transformed to the address 2164 by selecting digits in positions 3 to 6 and reversing their order. Digit positions having the most uniform distributions are selected. This hashing transformation technique has been used in conjunction with static key sets.

OVERFLOW HANDLING: (or) Collision-Resolution Technique:

Two techniques open addressing and chaining are used to detect collisions and overflows.

The general objective of a collision-resolution technique is to attempt to place colliding records elsewhere in the table. This requires the investigation of a series of table positions until an empty one is found to accommodate a colliding record.

OPEN ADDRESSING:

Here, collisions are simply resolved by computing a sequence of hash slots. Two types of techniques :

- 1) Linear Probing
- 2) Quadratic Probing

1. Linear Probing:

Here, the function f is defined as $f(i) = i$. It indicates that whenever we encounter collisions, the next available cell is searched sequentially and data elements are placed accordingly.

The following figure shows a hash table with seven locations(buckets) numbered from 0 to 6. Here the divisor we use is 7. Initially, we insert 23 and its position is 2 as $23 \% 7 = 2$.

0	1	2	3	4	5	6
		23				

Next, we insert 50 and its position is 1 and the arrangement is as follows.

0	1	2	3	4	5	6
	50	23				

Then we insert 30 and its position is 2, but the bucket number 2 is already occupied by 23. So, collision has occurred. Therefore, the value 30 gets next available cell, which is 3. The orientation is as follows.

0	1	2	3	4	5	6
	50	23	30			

Similarly, when we wish to insert 38, and we face collision again. Now it is placed at the next available cell, which is 4. The arrangement of data elements is as follows.

0	1	2	3	4	5	6
	50	23	30	38		

Overhead in this technique is the time taken for finding the next available cell.

2. Quadratic Probing:

In this case, when the collision occurs at hash address h , then this method searches the table at location $h+1$, $h+2$, and $h+9$. The ^{hash} function will now be defined as

$$(h(x) + i^2) \% \text{hash size}$$

Now let us consider a table of size 10 and index numbered from 0 to 9. Initially the table looks as follows.

0	1	2	3	4	5	6	7	8	9

When we wish to insert 23, we can easily insert at location 3 as shown below.

0	1	2	3	4	5	6	7	8	9
			23						

Next we want to insert 81 and it is easily placed at location 1.

0	1	2	3	4	5	6	7	8	9
	81		23						

Now we want to insert 93 and as the position 3 is already occupied, collision takes place. So, the cell with distance one apart is checked and if it is free then the new data element is placed, which is as shown below.

0	1	2	3	4	5	6	7	8	9
	81		23	93					

Now we wish to insert 113, in this case the position 3 and 4 are already occupied, so the cell with distance 4 is checked and it is found empty then the new value is placed at location 7.

0	1	2	3	4	5	6	7	8	9
	81		23	93			113		

CHAINING:

One of the reasons Open addressing and its variations perform poorly is that searching for an identifier involves comparison of identifiers with different hash values.

Many of the comparisons being made could be saved if we maintained lists of identifiers, one list per bucket, each list containing all the synonyms for that bucket. If this were done, a

search would then involve computing the hash address $f(X)$ and examining only those identifiers in the list for $f(X)$.

SORTING

The main classifications of sorting based on the source of data are

- a. Internal sorting
- b. External sorting
- c. Stable Sorting

- **External sorting**

External sorting is a process of sorting in which large blocks of data stored in storage devices are moved to the main memory and then sorted .i.e A sort can be external if the records that it is sorting are in Auxiliary storage.

- **Internal sorting**

Internal sorting is a process of sorting the data in the main memory. i.e A sort can be Internal if the records that it is sorting are in main memory.

- **Stable sort**

A sorting technique is called stable if for all records i and j such that $k[i]$ equals $k[j]$, if $r[i]$ precedes $r[j]$ in the original file, $r[i]$ precedes $r[j]$ in the sorted file. That is, a stable sort keeps records with the same key in the same relative order that they were in before the sort.

Various factors to be considered in deciding a sorting algorithm

- a. Programming time
- b. Execution time of the program
- c. Memory needed for program environment

SELECTION SORT

- Scan the entire list. Find the smallest element and exchange it with the first element, putting the smallest element in its final position (first) in the sorted list.
- Then scan the list, starting from the second element.

- Find the smallest element among the last $n-1$ elements and exchange it with the second element, putting the second smallest element in its final position.

$$A_0 \leq A_1 \leq \dots \leq A_{i-1}$$

ALGORITHM SELECTION SORT ($A[0..n-1]$)

```
for i = 0 to n-2 do
    min = i
    for j = i+1 to n-1 do
        if  $A[j] < A[\text{min}]$ 
            min = j
    swap  $A[i]$  and  $A[\text{min}]$ 
```

An example, the action of the algorithm on the list 89,45,68,90,29,34,17 is illustrated as

```
i=0 | 89 45 68 90 29 34 17
i=1 17 | 45 68 90 29 34 89
i=2 17 29 | 68 90 45 34 89
i=3 17 29 34 | 90 45 68 89
i=4 17 29 34 45 | 90 68 89
i=5 17 29 34 45 68 | 90 89
i=6 17 29 34 45 68 89 | 90
```

- The Selection sort finds the minimum element in the "unsorted" part of the array.
- The difference between Selection sort and Bubble sort is that, Bubble sort selects the maximum among remaining elements at each stage.
- Selection sort is quadratic in both the worst and the average case, and requires no extra memory.

Analysis

Thus, selection sort is an $\Theta(n^2)$ algorithm on all inputs.

BUBBLE SORT

- Compare the adjacent elements in the list and exchange them if they are out of order.
- By doing it repeatedly, we end up the “bubbling up” the largest element to the last position, the n^{th} position on the list.
- In the next pass, scan the list from the first element to the $n-1$ elements, comparing adjacent elements and bubbling up the second largest element to the $n-1^{\text{th}}$ position.
- The next pass bubbles up the second largest element, and so on until, after $n-1$ passes, the list is sorted.

ALGORITHM

```

bubble sort(a[0..n-1])
    for i = 0 to n-1 do
        for j = 0 to n-2-i do
            if A[j+1] < A[j]
                swap A[j] and A[j+1]
    
```

The action of the algorithm on the list 89,45,68,90,29,34,17 is illustrated as

PASS 1	89	-	45		68		90		29		34		17
	45		89	-	68		90		29		34		17
	45		68		89	-	90	-	29		34		17
	45		68		89		29		90	-	34		17
	45		68		89		29		34		90	-	17
	45		68		89		29		34		17		90

PASS 2	45	-	68	-	89	-	29		34		17		90
	45		68		29		89	-	34		17		90
	45		68		29		34		89	-	17		90
	45		68		29		34		17		89		90

Analysis

Thus, Bubble sort is an $\Theta(n^2)$ algorithm on all inputs.

INSERTION SORT

- One of the simplest sorting algorithms is the *insertion sort*.
- If the first few objects are already sorted, an unsorted object can be inserted in the sorted set in proper place. This is called insertion sort.
- The algorithm considers the elements one at a time, inserting each in its suitable place among those already sorted.
- Instead of inserting an element anywhere in the list and resorting it again, each time when a new element is encountered, it is inserted in the correct position.
- Insertion sort consists of $n - 1$ passes. For pass $p = 2$ through n , insertion sort ensures that the elements in positions 1 and p (i.e 2) are in sorted order.
- In pass p , move the p th element left until its correct place is found among the first p elements.

Original 34 | 8 64 51 32 21

Insertion sort after each pass

After $p = 2$ 8 34 | 64 51 32 21

After $p = 3$ 8 34 64 | 51 32 21

After $p = 4$ 8 34 51 64 | 32 21

After $p = 5$ 8 32 34 51 64 | 21

After $p = 6$ 8 21 32 34 51 64 |

ALGORITHM insertionsort((a[0..n-1])

 for $i = 1$ to $n-1$ do

$v = A[i]$

$j = i-1$

 while $j \geq 0$ and $A[j] > v$ do

$A[j+1] = A[j]$

$j = j - 1$

$A[j+1] = v$

Analysis of Insertion Sort

Because of the nested loops, each of which can take n iterations, insertion sort is $\Theta(n^2)$.

SHELLSORT

- Shell sort, named after its inventor, Donald Shell, was one of the first algorithms to break the quadratic time barrier.
- It works by comparing elements that are distant; the distance between comparisons decreases as the algorithm runs until the last phase, in which adjacent elements are compared. For this reason, Shell sort is sometimes referred to as *diminishing increment sort*.
- Shell sort uses a sequence, $h_1, h_2, \dots, h_t$, called the *increment sequence*. Any increment sequence will do as long as $h_1 = 1$, but obviously some choices are better than others.
- All elements spaced h_k apart are sorted. The file is then said to be h_k -sorted.
- For example, Figure shows an array after several phases of Shell sort.
- Shellsort routine using Shell's increments (better increments are possible)

Original 81 94 11 96 12 35 17 95 28 58 41 75 15

After 5-sort 35 17 11 28 12 41 75 15 96 58 81 94 95

After 3-sort 28 12 11 35 15 41 58 17 94 75 81 96 95

After 1-sort 11 12 15 17 28 35 41 58 75 81 94 95 96

Shell sort after each pass

```
void shellsort( int A[ ], int n)
{
    int i, j, k, temp;
    for( k = n/2; k > 0; k /= 2 )
        for( i = k; i < n; i++ )
```

```
{  
    temp = A[i];  
    for( j = i; j >= k && A[j-k] > temp; j = j - k )  
        A[j] = A[j-k];  
    A[j] = temp;  
}  
}
```

Example

Worst-Case Analysis of Shellsort

The worst case time complexity of shell sort is $\Theta(n^2)$.

RADIX SORT

- Radix sort is a small method that many people intuitively use when alphabetizing a large list of names.
- Specifically, the list of names is first sorted according to the first letter of each names, that is, the names are arranged in 26 classes

- Intuitively, one might want to sort numbers on the most significant digit, but Radix sort on the other hand sorts based on the least significant digits first.
- On the first pass entire numbers are sorted on the least significant digit and combined in an array.
- Then on the second pass, the entire numbers are sorted again on the second least-significant digits and combine in an array and so on. Following example shows how Radix sort operates on seven 3-digits number.

Buckets	INPUT	1 st Pass	2 nd pass	3 rd Pass
1.	329	720		
2.	457		329 720	
3.	657		839 436	329 355
4.	839			436 457
5.	436	355	457 647 355	
6.	720	436		647
7.	355	457 657		720
8.				839
9.		329 839		

- In the above example, the first column is the input. The remaining shows the list after successive sorts on increasingly significant digits position.
- The code for Radix sort assumes that each element in the n -element array A has d digits, where digit 1 is the lowest-order digit and d is the highest-order digit.

RADIX_SORT (A, d)

for $i \leftarrow 1$ to d do

 use a stable sort to sort A on digit i

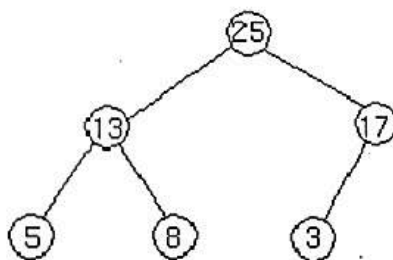
 // counting sort will do the job

Analysis

The running time depends on the stable used as an intermediate sorting algorithm. When each digit is in the range 1 to k , and k is not too large, COUNTING_SORT is the obvious choice. In case of counting sort, each pass over n d -digit numbers takes $O(n + k)$ time. There are d passes, so the total time for Radix sort is $O(dn + kd)$. When d is constant and $k = O(n)$, the Radix sort runs in linear time.

HEAP SORT

- The binary heap data structure is an array that can be viewed as a complete binary tree.
- Each node of the binary tree corresponds to an element of the array. The array is completely filled on all levels except possibly lowest.



- Heaps are represented in level order, going from left to right. The array corresponding to the heap above is [25, 13, 17, 5, 8, 3].

The root of the tree $A[1]$ and given index i of a node, the indices of its parent, left child and right child can be computed

PARENT (i)

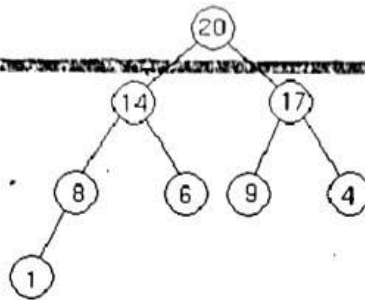
return floor($i/2$)

LEFT (i)

return $2i$

RIGHT (i)

return $2i + 1$



is represented by the array $\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ [20, & 14, & 17, & 8, & 6, & 9, & 4, & 1]. \end{matrix}$

- Go from the 20 to the 6 first. The index of the 20 is 1. To find the index of the left child, calculate $1 * 2 = 2$. This takes us to 14.
- Now, to go right, calculate $2 * 2 + 1 = 5$. This takes us to the 6.
- Try going from 4 to 20. 4's index is 7. To go to the parent, calculate $7 / 2 = 3$, which takes us to the 17.
- Now, to get 17's parent, calculate $3 / 2 = 1$, which takes us to the 20.

✓ **Structure Property**

A heap is a binary tree that is completely filled, with the possible exception of the bottom level, which is filled from left to right. Such a tree is known as a *complete binary tree*. Figure shows an example.

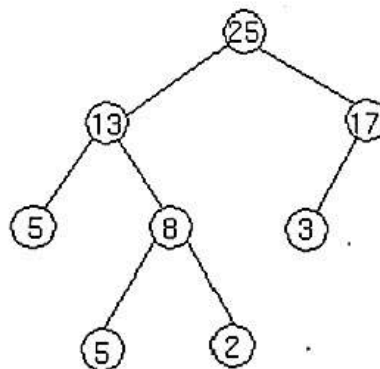
✓ **Heap Order Property**

This property allows operations to be performed quickly is the heap order property.

Min-heap: If the minimum element has to be found quickly, the smallest element should be at the root. If the subtrees should also be min-heaps, then all the sub root nodes should be smaller than all of their descendants.

Max-heap : If the maximum has to be found quickly, the largest element should be at the root. If the subtrees should also be max-heaps, then all the sub root nodes should be larger than all of their descendants.

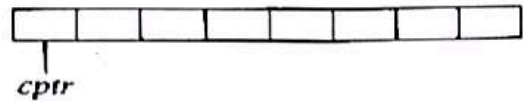
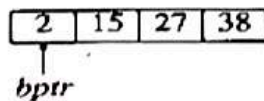
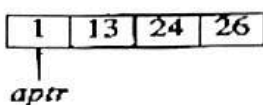
- A heap of height h has the minimum number of elements when it has just one node at the lowest level.
- The levels above the lowest level form a complete binary tree of height $h-1$ and 2^h-1 nodes.
- Hence the minimum number of nodes possible in a heap of height h is 2^h .
- Clearly a heap of height h , has the maximum number of elements when its lowest level is completely filled.
- In this case the heap is a complete binary tree of height h and hence has $2^{h+1}-1$ nodes. Following is not a heap, because the heap order property holds – but the structure property does not hold.



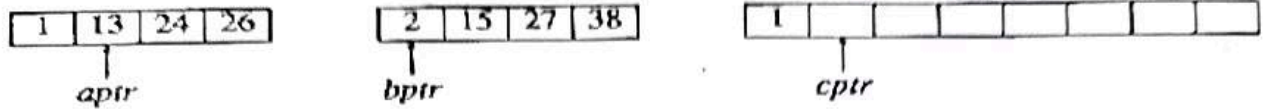
Algorithm

MERGE SORT

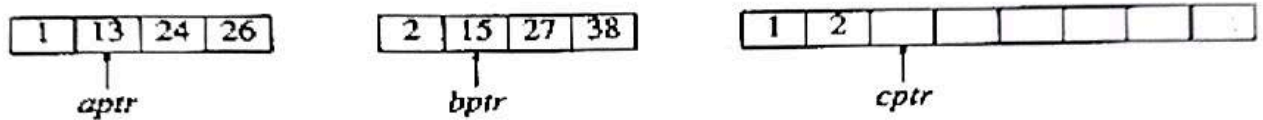
- Merge sort is another example of divide-and-conquer
- Given a sequence of 'n' elements $a[1], \dots, a[n]$. The general idea is to split them into 2 sets $a[1], \dots, a[n/2]$ and $a[[n/2]+1], \dots, a[n]$.
- Splitting continues until a set with only one element is encountered.
- Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of 'n' elements.
- The first two elements are compared and sorted and put into an array of size 2. Thus elements are compared and sorted in pairs from $a[1]$ to $a[n]$.
- In the next step, sets containing 2 elements are compared and sorted. This continues until the entire list is sorted.
- Thus, merge sort is other ideal example of the divide-and-conquer strategy in which the splitting is into 2 equal-sized sets & the combining operation is the merging of 2 sorted sets into one.
- The fundamental operation in this algorithm is merging two sorted lists. Because the lists are sorted, this can be done in one pass through the input, if the output is put in a third list.
- The basic merging algorithm takes two input arrays 'a' and 'b', an output array 'c', and three counters, *aptr*, *bptr*, and *cptr*, which are initially set to the beginning of their respective arrays.
- The smaller of $a[\text{aptr}]$ and $b[\text{bptr}]$ is copied to the next entry in 'c', and the appropriate counters are advanced.
- When either input list is exhausted, the remainder of the other list is copied to 'c'.
- An example of how the merge routine works is provided for the following input.



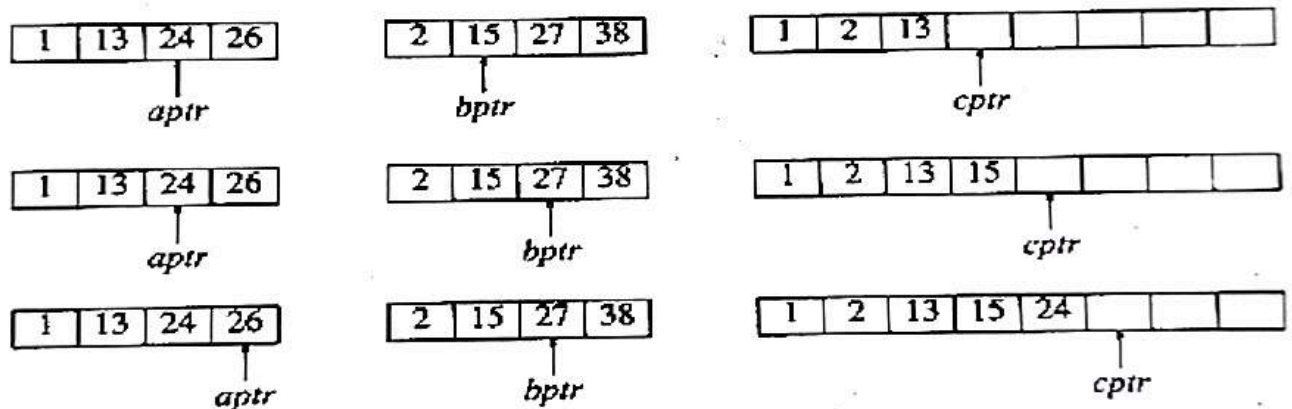
- If the array 'a' contains 1, 13, 24, 26, and b contains 2, 15, 27, 38, then the algorithm proceeds as follows: First, a comparison is done between 1 and 2. 1 is added to 'c', and then 13 and 2 are compared.



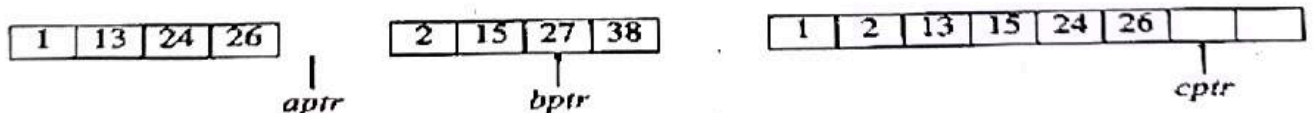
- 2 is added to 'c', and then 13 and 15 are compared.



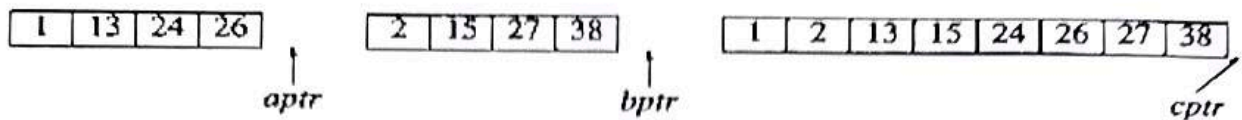
- 13 is added to 'c', and then 24 and 15 are compared. This proceeds until 26 and 27 are compared.



- 26 is added to c, and the 'a' array is exhausted.



- The remainder of the 'b' array is then copied to 'c'.



- The time to merge two sorted lists is clearly linear, because at most $n - 1$ comparisons are made, where n is the total number of elements. Every comparison adds an element to c , except the last comparison, which adds at least two.
- If $n = 1$, there is only one element to sort, and the answer is at hand. Otherwise, recursively mergesort the first half and the second half. This gives two sorted halves, which can then be merged together using the merging algorithm described above.
- For instance, to sort the eight-element array 24, 13, 26, 1, 2, 27, 38, 15, recursively sort the first four and last four elements, obtaining 1, 13, 24, 26, 2, 15, 27, 38.
- Then merge the two halves as above, obtaining the final list 1, 2, 13, 15, 24, 26, 27, 38. This algorithm is a classic divide-and-conquer strategy. The problem is divided into smaller problems and solved recursively.
- The conquering phase consists of patching together the answers. Divide-and-conquer is a very powerful use of recursion.

Algorithm MergeSort(low,high)

```

{
    if (low < high) then //if there are more than one element
    {
        mid = [(low+high)/2]; //Divide P into subproblems. find where to split the set
        mergesort (low,mid); //solve the subproblems.
        mergesort(mid+1,high);
        merge(low,mid,high); //combine the solutions .
    }
}
    
```

Merging 2 sorted subarrays using auxiliary storage.

```
Algorithm merge(low,mid,high)
{
h=low; i=low; j=mid+1;
while ((h<=mid) and (j<=high)) do
{
if (a[h]<=a[j]) then
{
b[i]=a[h];
h = h+1;
}
else
{
b[i]= a[j];
j=j+1;
}
i=i+1;
}
if (h>mid) then
for k=j to high do
{
b[i]=a[k];
i=i+1;
}
else
for k=h to mid do
{
b[i]=a[k];
i=i+1;
}
for k=low to high do a[k] = b[k];
```

}

Analysis :

Its worst case complexity is $O(n \log n)$.

QUICK SORT

- The divide-and-conquer approach can be used to arrive at an efficient sorting method different from merge sort. In merge sort, the file $a[1:n]$ was divided at its midpoint into sub arrays which were independently sorted & later merged.
- In Quick sort, the division into 2 sub arrays is made so that the sorted sub arrays do not need to be merged later.
- This is accomplished by rearranging the elements in $a[1:n]$ such that $a[i] \leq a[j]$ for all i between 1 & m and all j between $(m+1)$ & n for some m , $1 \leq m \leq n$.
- Thus the elements in $a[1:m]$ & $a[m+1:n]$ can be independently sorted.
- No merge is needed. This rearranging is referred to as partitioning.
- Function partition of algorithm accomplishes an in-place partitioning of the elements of the elements $a[m:p-1]$.
- It is assumed that $a[p] \geq a[m]$ and that $a[m]$ is the partitioning element. If $m=1$ & $p-1=n$, then $a[n+1]$ must be defined and must be greater than or equal to all elements in $a[1:n]$
- The assumption that $a[m]$ is the partition element is merely for convenience, other choices for the partitioning element than the first item in the set are better in practice.
- As its name implies, *quicksort* is the fastest known sorting algorithm in practice. Its average running time is $O(n \log n)$.
- It is very fast, mainly due to a very tight and highly optimized inner loop.
- It has $O(n^2)$ worst-case performance, but this can be made exponentially unlikely with a little effort. The basic algorithm to sort an array S consists of the following four easy steps:
 - Let l and r be two pointers pointing to the first and the last element in the list.

Quick sort is a sorting algorithm that uses the divide and conquers strategy. In this method division is dynamically carried out. The three steps of quick sort are as follows:

Divide: Split the array into two sub arrays that each element in the left sub array is less than or equal to the middle element and each element in the right sub array is greater than the middle element. The splitting of the array into two sub arrays is based on the pivot element all the elements that are more than pivot should be in the right sub array.

Conquer: Recursively sort the two sub arrays

Combine: Combine the sorted elements in a group to form a list of sorted elements.

STEPS:

1. If the number of elements in S is 0 or 1, then return.
2. Compare the first two elements. Choose the largest as the pivot. In case the first two elements are the same, compare the second and the third and choose the largest. Continue this till the last element is reached. In case all the elements have the same value, a pivot cannot be chosen. Return.
3. Swap $A[l]$ and $A[r]$. The function interchange (a, i, j) does this.
4. Advance the 'l' pointer till an element greater than the pivot element is found.
5. Decrement 'r' pointer until an element less than or equal to the pivot is found.
6. Check whether $l < r$. If so, swap $a[l]$ and $a[r]$.
7. If $l > r$, partition the list into 2 parts. (upto $A[r]$ and after $A[r]$).
8. Follow the above mentioned steps for each partition.

EXAMPLE

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
5	3	1	9	8	2	4	7
	i						j
5	3	1	9	8	2	4	7
	i						j
5	3	1	9	8	2	4	7
		i				j	
5	3	1	4	8	2	9	7
			i			j	
5	3	1	4	8	2	9	7
			i	j			

5	3	1	4	2	8	9	7
5	3	1	4	^j 2	ⁱ 8	9	7
2	3	1	4	5	8	9	7

Then Left half and right half is sorted individually by following the same procedure.

Analysis :

Its worst case complexity is $O(n \log n)$.

EXTERNAL SORTING

- All the algorithms examined above require that the input fit into main memory.
- There are, however, applications where the input is much too large to fit into memory.
- External sorting algorithms are designed to handle very large inputs.
- Most of the internal sorting algorithms take advantage of the fact that memory is directly addressable.
- Shellsort compares elements $a[i]$ and $a[i - hk]$ in one time unit.
- Heapsort compares elements $a[i]$ and $a[i * 2]$ in one time unit.
- Quicksort with median-of-three partitioning, requires comparing $a[\text{left}]$, $a[\text{center}]$, and $a[\text{right}]$ in a constant number of time units.
- If the input is on a tape, then all these operations lose their efficiency, since elements on a tape can only be accessed sequentially.
- Even if the data is on a disk, there is still a practical loss of efficiency because of the delay required to spin the disk and move the disk head.
- To see how slow external accesses really are, create a random file that is large, but not too big to fit in main memory.
- Read the file in and sort it using an efficient algorithm.
- The time it takes to sort the input is certain to be insignificant compared to the time to read the input, even though sorting is an $O(n \log n)$ operation and reading the input is only $O(n)$.

Model for External Sorting

- The wide variety of mass storage devices makes external sorting much more device dependent than internal sorting.
- The algorithms work on tapes, which are probably the most restrictive storage medium.
- Since access to an element on tape is done by winding the tape to the correct location, tapes can be efficiently accessed only in sequential order (in either direction).
- Assume that there are at least three tape drives to perform the sorting.
- Two drives are needed to do an efficient sort; the third drive simplifies matters.
- If only one tape drive is present, then any algorithm will require (n^2) tape accesses.

The Simple Algorithm

- The basic external sorting algorithm uses the merge routine from mergesort.
- Suppose there are four tapes, T_{a1} , T_{a2} , T_{b1} , T_{b2} , which are two input and two output tapes.
- Depending on the point in the algorithm, the a and b tapes are either input tapes or output tapes. Suppose the data is initially on T_{a1} .
- Suppose further that the internal memory can hold (and sort) m records at a time.
- A natural first step is to read m records at a time from the input tape, sort the records internally, and then write the sorted records alternately to T_{b1} and T_{b2} .
- Call each set of sorted records a run. When this is done, rewind all the tapes.

T_{a1}	81	94	11	96	12	35	17	99	28	58	41	75	15
T_{a2}													
T_{b1}													
T_{b2}													

- If $m = 3$, then after the runs are constructed, the tapes will contain the data indicated in the following figure.
- Now T_{b1} and T_{b2} contain a group of runs.
- Take the first run from each tape and merge them, writing the result, which is a run twice as long, onto T_{a1} .
- Then take the next run from each tape, merge these, and write the result to T_{a2} .
- Continue this process, alternating between T_{a1} and T_{a2} , until either T_{b1} or T_{b2} is empty.

- At this point either both are empty or there is one run left.
- In the latter case, copy this run to the appropriate tape.
- Rewind all four tapes, and repeat the same steps, this time using the tapes as input and the b tapes as output.
- This will give runs of 4m. Continue the process until a run of length n is obtained.

T_{a1}	11	12	35	81	94	96	15
T_{a2}	17	28	41	58	75	99	
T_{b1}							
T_{b2}							

T_{a1}	11 12 17 28 35 51 58 75 81 94 96 99											
T_{a2}												
T_{b1}												
T_{b2}												

T_{a1}	11	12	15	17	28	35	41	58	75	81	94	96	99
T_{a2}													
T_{b1}													
T_{b2}													

Multiway Merge

- If there are extra tapes, then the number of passes required to sort our input can be reduced.
- This can be done by extending the basic (two-way) merge to a k-way merge.
- Merging two runs is done by winding each input tape to the beginning of each run.
- Then the smaller element is found, placed on an output tape, and the appropriate input tape is advanced. If there are k input tapes, this strategy works the same way, the only difference being that it is slightly more complicated to find the smallest of the k elements.
- The smallest of these elements can be found by using a priority queue.
- To obtain the next element to write on the output tape, delete_min operation is performed.
- The appropriate input tape is advanced, and if the run on the input tape is not yet completed, e insert the new element into the priority queue.
- Using the same example as before, distribute the input onto the three tapes.

T_{a1}						
T_{a2}						
T_{a3}						
T_{b1}	11	81	94	41	58	75
T_{b2}	12	35	96	15		
T_{b3}	17	28	99			

Two more passes of three-way merging are needed to complete the sort.

T_{a1}	11	12	17	28	35	81	94	96	99	
T_{a2}	15	41	58	75						
T_{a3}										
T_{b1}										
T_{b2}										
T_{b3}										

T_{a1}												
T_{a2}												
T_{a3}												
T_{b1}	11	12	15	17	28	35	41	58	75	81	94	96
T_{b2}												
T_{b3}												

Polyphase Merge

- The k-way merging strategy developed in the last section requires the use of $2k$ tapes.
- This could be prohibitive for some applications. It is possible with only $k + 1$ tapes.
- As an example, two-way merging using only three tapes is shown.
- Suppose there are three tapes, T_1 , T_2 , and T_3 , and an input file on T_1 that will produce 34 runs.

- One option is to put 17 runs on each of T_2 and T_3 . This result could be merged onto T_1 , obtaining one tape with 17 runs.
- The problem is that since all the runs are on one tape, put some of these runs on T_2 to perform another merge.
- The logical way to do this is to copy the first eight runs from T_1 onto T_2 and then perform the merge.
- This has the effect of adding an extra half pass for every pass.
- An alternative method is to split the original 34 runs unevenly. Suppose 21 runs are put on T_2 and 13 runs on T_3 . Then merge 13 runs onto T_1 before T_3 becomes empty.
- At this point, rewind T_1 and T_3 , and merge T_1 , with 13 runs, and T_2 , which has 8 runs, onto T_3 . Then merge 8 runs until T_2 becomes empty, which would leave 5 runs left on T_1 and 8 runs on T_3 .
- Merge T_1 and T_3 , and so on. The following table below shows the number of runs on each tape after each pass.

Run	After	After	After	After	After	After	After
Const.	$T_3 + T_2$	$T_1 + T_2$	$T_1 + T_3$	$T_2 + T_3$	$T_1 + T_2$	$T_1 + T_3$	$T_2 + T_3$

T_1	0	13	5	0	3	1	0	1
T_2	21	8	0	5	2	0	1	0
T_3	13	0	8	3	0	2	1	0

- The original distribution of runs makes a great deal of difference. For instance, if 22 runs are placed on T_2 , with 12 on T_3 , then after the first merge, 12 runs on T_1 and 10 runs on T_2 are obtained.
- After another merge, there are 10 runs on T_1 and 2 runs on T_3 .

- At this point the going gets slow, because only two sets of runs can be merged before T_3 is exhausted.
 - Then T_1 has 8 runs and T_2 has 2 runs. Again, merge two sets of runs, obtaining T_1 with 6 runs and T_3 with 2 runs.
 - After three more passes, T_2 has two runs and the other tapes are empty.
 - Copy one run to another tape, and then finish the merge. It turns out that the first distribution obtained is optimal.
 - If the number of runs is a Fibonacci number F_n , then the best way to distribute them is to split them into two Fibonacci numbers F_{n-1} and F_{n-2} . Otherwise, it is necessary to pad the tape with dummy runs in order to get the number of runs up to a Fibonacci number.
-

INTRODUCTION

The representation of a particular data structure in the memory of a computer is called a **storage structure**.

A storage structure representation in auxiliary memory is often called a **file structure**.

EXTERNAL STORAGE DEVICES

A device other than main memory, data can be stored and retrieved for processing at some subsequent point of time.

- Storage and retrieval operations are nothing but writing and reading operations respectively.
- They provide large capacity at a lower cost.
- Primary uses:
 - Backup of programs during execution.
 - Store program and subprogram for later reference.
 - Storage of information in “files”.
- Some of the most common ones are magnetic tapes, drums, and disk drives.

MAGNETIC TAPE

- Magnetic tape is a plastic medium coated with magnetized ferrite substance.
- Information encoded-character by character.
- Binary code represents character.
- Additional bit called parity bit included avoiding redundancy.

- Read/write information at the rate of 125 inches/sec & data transfer rate at 200000 bytes/sec.
- During write cycle, certain length of tape is left as gap, between two record gap called interrecord gap.
- This reduces the storage area. E.g., $\frac{1}{2}$ inch $\rightarrow$ $\frac{1}{2}$ capacity for storage of information.
- Records are grouped into blocks without interrecord gap.
- Gap between two blocks is known as interlock gap.
- Storage capacity increased by blocking factors.

LIMITATION:

- Records must be processed in the order in which they reside in the tape.
- Accessing record requires scanning of all records.
- Easy to place or remove from a tape drive and cheap.

MAGNETIC DRUM

- Magnetic drum is a metallic cylinder-10 to 36 inches in diameter. Coated externally with magnetic recording material.
- Cylindrical surface, divided into parallel bands called tracks.
- Tracks subdivided into smallest addressable units called sector or block by nature.
- Sectors or blocks do accessing of any block of drum. Hence the drum is called as direct-access-storage device.
- Sectors-fixed length –arcs of a track, with same size.
- May have, fixed head-one for each track
Movable head-group by heads mounted on a rail.

- Fixed head drum:

Time to access a certain location called latency (L) or rotationary delay.

- Movable head drum: additional time required to move head to a desired track called seek time.
- Access time, $A(i) = L(i) + S(i)$.
- Time required completing operation called transition time or flow time also depends on speed of rotation.
$$T(i) = L(i) + S(i) + T(i)$$

For fixed head drum $S(i) = 0$.

- Average time $T'(i) = S'(i) + L'(i) + T(i) = A'(i) + T(i)$.
- Fixed-head drum gives fast access, time & data transfer rate.
- Movable head drum, large storage capacity & storing large volumes of data.
- Can't be removed from shaft or drive.

MAGNETIC DISK

- Provides lower access time & high speed data transfer

- Type: fixed and exchangeable disks.
- Pack consists of a number of metallic plates, stacked on a spindle.
- Both surfaces coated with ferromagnetic materials.
- Surface divided into concentric bands called tracks.
- Recording density of inner track is greater than outer track.
- Storage and data transfer occurs without using head is called cylinder or seek area.
- Each track has its own read/write head for fast access.
- Heads are attached to a movable arm; during data transfer whole assembly moves to point the position.
- In 1973, IBM revolutionized Winchester technology in its model 3340 disk memory.
- Entire disks and heads are sealed as clean chamber damaged we can remove entire chamber.
- Rotational delay is the time taken to wait for the disk to rotate and transfer can commence.
- We can read 4,096 bytes from IBM 3330 magnetic disk.

FLOPPY DISK

- Mylar plastic-usually 5 1/2 or 8 inches in dia.
Coating-magnetic material.
- 8 to 26 sectors/track with 128 to 512 bytes.
- Capacity b/w 125 KB to 1 MB and transmission rate is 5 to 10 characters/msec.

MASS STORAGE DEVICE

- Card/strip devices, capacity of one-half billion characters.
- Principle operation, selection of card from group & transportation of this card to a capstan.
- Access time greater than 1-1/2 seconds.
- Supports large capacity at low cost/bit.
- Gives third level storage facility.
- More realistic to store copies of "often accessed" programs.

INTERMEDIATE STORAGE DEVICES

- Electronic disk called electronic.
- Fast accessing time without machine movement.
- Used to fill the gaps b/w main memory and direct access storage.
- Acceptable electronic disk is charge-coupled device.
- Based on semiconductor technology, provides average access time of 60 microsecond.